

Pseudocode Fundamentals For Program Planning

Generated Jul 8, 2026 2:38 AM

What We'll Learn

By the end of this lesson, students can write simple pseudocode to plan a program before coding.

Image unavailable: images/pseudocode-planning-example.svg

Words We'll Use

- Pseudocode
- Algorithm
- Program planning
- Sequence
- Input
- Process
- Output

Before We Start

Before writing code, how can we plan the steps of a program so it is easier to understand?

Let's Understand

Pseudocode is a simple way to write program steps using plain language.

It is not written using strict programming syntax. Instead, it helps us plan what the program should do before we write the actual code.

A good pseudocode should be:

- Clear
- Step-by-step
- Easy to follow
- Written in the correct order
- Focused on logic, not programming syntax

Common pseudocode action words:

- START - begins the program
- INPUT - gets data from the user
- SET - stores or assigns a value
- COMPUTE - performs a calculation
- DISPLAY - shows output to the user
- END - ends the program

Basic pseudocode structure:

```
START
INPUT value
PROCESS the value
DISPLAY result
END
```

Let's Look at Examples

Example 1

Problem:

Write pseudocode that displays "Hello, Student!"

Pseudocode:

```
START  
  DISPLAY "Hello, Student!"  
END
```

Explanation:

The program starts, displays the message, and ends.

Example 1: Display a Message

This example shows how to write pseudocode for a program that displays a simple message.

Problem:

Create pseudocode that displays the message "Welcome to Computer Programming."

Pseudocode:

```
START
DISPLAY "Welcome to Computer Programming"
END
```

Explanation:

The program starts, displays a message, and ends.

Common mistake

Students may forget to include START and END.

Mini-check

What action word is used to show a message?

Example 2

Problem:

Write pseudocode that asks for a name and displays it.

Pseudocode:

```
START
INPUT name
  DISPLAY name
END
```

Explanation:

The program asks for the user's name, then displays the same name.

Example 2: Input a Name

This example shows how to write pseudocode that asks the user for input.

Problem:

Create pseudocode that asks for the user's name and displays it.

Pseudocode:

```
START
INPUT name
DISPLAY name
END
```

Explanation:

The program asks the user to enter a name, then displays the entered name.

Common mistake

Students may use `DISPLAY name` before using `INPUT name`.

Mini-check

Why should `INPUT name` come before `DISPLAY name`?

Example 3

Problem:

Write pseudocode that asks for two numbers, adds them, and displays the sum.

Pseudocode:

```
START
  INPUT num1
  INPUT num2
  COMPUTE sum = num1 + num2
  DISPLAY sum
END
```

Explanation:

The program receives two numbers, computes their sum, and displays the result.

Example 3: Add Two Numbers

This example shows how pseudocode can be used to plan a simple calculation.

Problem:

Create pseudocode that asks for two numbers, adds them, and displays the sum.

Problem Analysis:

Input: firstNumber, secondNumber Process: $\text{sum} = \text{firstNumber} + \text{secondNumber}$ Output: Display sum

Pseudocode:

```
START
INPUT firstNumber
INPUT secondNumber
COMPUTE sum = firstNumber + secondNumber
DISPLAY sum
END
```

Explanation:

The program receives two numbers, computes their sum, and displays the result.

Common mistake

Students may display the answer without computing the sum first.

Mini-check

What variable stores the answer?

Example 4: Compute the Average of Three Scores

This example shows how to use pseudocode for a formula.

Problem:

Create pseudocode that asks for three quiz scores, computes the average, and displays the average.

Problem Analysis:

Input: quiz1, quiz2, quiz3 Process: $\text{average} = (\text{quiz1} + \text{quiz2} + \text{quiz3}) / 3$ Output: Display average

Pseudocode:

```
START
INPUT quiz1
INPUT quiz2
INPUT quiz3
COMPUTE average = (quiz1 + quiz2 + quiz3) / 3
DISPLAY average
END
```

Explanation:

The program gets three quiz scores, adds them, divides the total by 3, and displays the average.

Common mistake

Students may forget to use parentheses in the formula.

Mini-check

Why do we divide by 3?

Example 5: Convert Flowchart Steps to Pseudocode

This example shows how flowchart logic can be written as pseudocode.

Flow:

Start → Input name → Display name → End

Pseudocode:

```
START
INPUT  name
DISPLAY name
END
```

Explanation:

The same logic from the flowchart is written using plain language steps.

Common mistake

Students may change the order of the steps when converting from flowchart to pseudocode.

Mini-check

Which step comes after INPUT name?

Example 6: Pseudocode vs Code

This example shows the difference between pseudocode and actual code.

Pseudocode:

```
START  
DISPLAY "Hello"  
END
```

Java Code:

```
System.out.println("Hello");
```

Explanation:

Pseudocode uses simple planning language, while Java code follows the rules and syntax of the Java programming language.

Common mistake

Students may write Java syntax when they are only asked to write pseudocode.

Mini-check

Which one is easier to understand for beginners?

Example 7: Real-Life Task as Pseudocode

This example shows that pseudocode can also be used to describe real-life step-by-step tasks.

Task:

Prepare for class.

Pseudocode:

```
START
Get notebook
Get pen
Check class schedule
Go to classroom
END
```

Explanation:

The task is written in a clear sequence from beginning to end.

Common mistake

Students may skip important steps, making the logic incomplete.

Mini-check

What step should come before going to the classroom?

Example 8: Input, Process, Output

This example shows how to identify input, process, and output before writing pseudocode.

Problem:

Create pseudocode that asks for the price of an item and the quantity bought, then computes and displays the total cost.

Problem Analysis:

Input: price, quantity Process: $\text{totalCost} = \text{price} * \text{quantity}$ Output: Display totalCost

Pseudocode:

```
START
INPUT price
INPUT quantity
COMPUTE totalCost = price * quantity
DISPLAY totalCost
END
```

Explanation:

The program gets the needed values, computes the total cost, and displays the answer.

Common mistake

Students may confuse input with output.

Mini-check

Is totalCost an input or a computed result?

Example 9: Arrange the Pseudocode

This example shows how to arrange pseudocode steps correctly.

Scrambled Steps:

```
DISPLAY total
INPUT number1
END
COMPUTE total = number1 + number2
START
INPUT number2
```

Correct Order:

```
START
INPUT number1
INPUT number2
COMPUTE total = number1 + number2
DISPLAY total
END
```

Explanation:

The program must start first, receive inputs, process the data, display the result, and then end.

Common mistake

Students may place `DISPLAY total` before computing the total.

Mini-check

What should happen first: input, process, or output?

Example 10: Student Introduction Program

This example shows a simple program that uses more than one input.

Problem:

Create pseudocode that asks for a student's name and grade level, then displays both details.

Problem Analysis:

Input: studentName, gradeLevel Process: none Output: Display studentName and gradeLevel

Pseudocode:

```
START
INPUT studentName
INPUT gradeLevel
DISPLAY studentName
DISPLAY gradeLevel
END
```

Explanation:

The program gets two pieces of information from the user and displays them.

Common mistake

Students may add a computation even when the problem does not require one.

Mini-check

Does this problem need a formula?

Example 11: Simple Salary Computation

This example shows pseudocode for computing salary.

Problem:

Create pseudocode that asks for the number of hours worked and the rate per hour, then computes and displays the salary.

Problem Analysis:

Input: hoursWorked, ratePerHour Process: $\text{salary} = \text{hoursWorked} * \text{ratePerHour}$ Output: Display salary

Pseudocode:

```
START
INPUT hoursWorked
INPUT ratePerHour
COMPUTE salary = hoursWorked * ratePerHour
DISPLAY salary
END
```

Explanation:

The program multiplies the number of hours worked by the rate per hour to compute the salary.

Common mistake

Students may use addition instead of multiplication.

Mini-check

What operation should be used to compute salary?

Example 12: Area of a Rectangle

This example shows how to write pseudocode using a basic math formula.

Problem:

Create pseudocode that asks for the length and width of a rectangle, then computes and displays its area.

Problem Analysis:

Input: length, width Process: $\text{area} = \text{length} * \text{width}$ Output: Display area

Pseudocode:

```
START
INPUT length
INPUT width
COMPUTE area = length * width
DISPLAY area
END
```

Explanation:

The program gets the length and width, multiplies them, and displays the area.

Common mistake

Students may forget to display the final answer.

Mini-check

What is the formula for the area of a rectangle?

Let's Practice

Write pseudocode for the following simple tasks:

1. Display your name.
2. Ask the user for their age.
3. Add two numbers.
4. Compute the average of three quiz scores.
5. Display a welcome message.

Now We Apply

Create pseudocode for a program that asks for a student's name and grade level, then displays a short introduction message.

Challenge Task

Create pseudocode for a program that asks for two numbers, computes their sum, difference, product, and quotient, then displays all results.

How Our Work Will Be Checked

- Correct use of START and END
- Correct use of action words
- Clear step-by-step order
- Complete input, process, and output
- Easy-to-understand logic

Let's Reflect

Why is it helpful to write pseudocode before creating the actual program code?

How We Show Learning

Students will submit pseudocode that solves a simple programming problem using correct order and clear action words.

Student Activities

Identify Pseudocode. Algorithm to Pseudocode. Complete Missing Lines. Trace with Values. Pseudocode Fix-It Task. Correction Log. Write pseudocode for area of rectangle. Write pseudocode for grade status. Exchange and revise with a partner.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.